Assignment No: 1

(1,1) (1,2) (1,3) (1,4)
* * * * → 1-4
(2,1) (2,2) (2,3) (2,4)
* * * * → 1-4
(3,1) (3,2) (3,3) (3,4)
* * * * → 1-4
* * * * → 1-4
(4,1) (4,2) (4,3) (4,4)

```
S.O.P("*");
for(int i=1; i≤4; i++){
    S.O.P("*");
}
```

* * * *

int i=1; i≤4 i++ i++
1≤4 2≤4 3≤4 4≤4 5≤4
↳ * ↳ * ↳ * ↳ * ↳ ×

Print/n → next line
Print →

1≤4 j=1
i=1 1≤4 2≤4 3≤4 4≤4 5≤4 ×
2≤4 * * * * ×
i=2 1≤4 2≤4 3≤4 4≤4 5≤4
* ↳ * * ↳ * ×
3≤4
i=3 1≤4 2≤4 3≤4 4≤4 5≤4 ×
4≤4 * ↳ * ↳ * ↳ *
i=4 1≤4 2≤4 3≤4 4≤4 5≤4 ×
* * * *

5≤4 ×
i=5 → out of loop.

Time Complexity

```
int n = 4
for(int i=1; i≤n; i++){        → [1,n]
    for(int j=1; j≤n; j++){    → [1,n]
```

initilization i = 1
condition i = n+1
incre i = n
initilization j = 4
condition = 4(n+1)
incre j = 4n

$(1+(n+1)+n)*(4+(4n+4)+4n)$
$(n+1)+n+5)+(4n+4)+4n$
$\cancel{(n+1)+(4n+4)+5n+5}$
$2n+$

$(1+n+1+n) * (4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2+16n+16n+16$

$16n^2+32n+16$

Remove constants

$16n^2+32n$

Remove lower value

$16n^2$

consider, only growth rate

$TC = O(n^2)$

$SC = O(1)$

Q.2

(1,1) (1,2) (1,3) (1,4)

1 1 1 1 → for(1-4)

(2,1) (2,2) (2,3) (2,4)

2 2 2 2 → for(1-4)

(3,1) (3,2) (3,3) (3,4)

3 3 3 3 → for(1-4)

4 4 4 4 → for(1-4)

(4,1) (4,2) (4,3) (4,4)

i=1 i≤4 int cnt = 1; $\boxed{1}$ cnt

↳ 1≤4 i++

↳ j=1 j≤4 j++

1≤4 2

↳ S.O.P(cnt) → 1

2≤4

↳ 1 3

3≤4 4

↳ 1

4≤4 5

↳ 1

5≤4 ×

cnt++

cnt=cnt+1

=1+1

=2

2

i≤4

↳ 1≤4 j++

↳ S.O.P(cnt) 2

↳ 2

2≤4 3

↳ 2

3≤4 4

↳ 2

4≤4 5

↳ 2

5≤4 ×

cnt++

cnt=cnt+1

=2+1

=3

i=2 2≤4

↳ j=1 j≤4 j++ i++

1≤4 2

↳ 3

2≤4 3

↳ 3

3≤4 4

↳ 3

4≤4 5

↳ 3

5≤4 ×

4

cnt=cnt+1

=3+1

=4

i=4 4≤4 j++

↳ j≤4 2

↳ 4

2≤4 3

↳ 4

3≤4 4

↳ 4

4≤4 5

↳ 4

5≤4 ×

i=5 5≤4 ×

5

Time complexity

int n = 4;

for(int i = 1; i≤n; i++), → [1, n]

for(int j = 1; j<=n; j++), → [1, n]

$(1+n+1+n)*(4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2+16n+16n+16$

$16n^2+32n+16$

Assignment No: 2

Q.1]

(1,1) (2,1) (3,1) (4,1) (5,1)

* * * * * → for(1-5)

(2,1) * (2,2) (2,3) (2,4) * (2,5) → for(1-5)

(3,1) * (3,2) (3,3) (3,4) * (3,5) → for(1-5)

(4,1) * (4,2) (4,3) (4,4) * (4,5) → for(1-5)

* * * * * (5,5) → for(1-5)

i=5 ← (5,1) (5,2) (5,3) (5,4) (5,5)

j=1 j=5 → i=1

* = (1,1) (1,2) (1,3) (1,4) (1,5)

j=1 ← (2,1) (3,1) (4,1) (5,1)

i=5 ← (5,1) (5,2) (5,3) (5,4) (5,5)

j=5 ← (2,5) (3,5) (4,5) (5,5)

i=1 1≤5

↳ 1≤5

↳ i==1 || j==1 || j==5 || j==5

↳ *

2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ * ↳ * ↳ *

2≤5

↳ 1≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ - ↳ - ↳ - ↳ *

3≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ - ↳ - ↳ - ↳ *

4≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ - ↳ - ↳ - ↳ *

5≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ * ↳ * ↳ * ↳ *

6≤5 ×

Time complexity int n=5; → [1,n]

```
for(int i=1; i<=n; i++){
    for(int j=1; j<=n; j++){
```

[1,n]

$(1+n+1+n) * (5 + 5n + 5 + 5n)$

$(2n+2) * (10n + 10)$

$20n^2 + 20n + 20n + 20$

$20n^2 + 40n + 20$

Remove constant

$20n^2 + 40n$

Remove lower value

$20n^2$

Consider, growth rate

$TC = O(n^2)$

$SC = O(1)$

Q.2]

(1,1) (1,2) (1,3) (1,4) (1,5)

* - - - * → for(1-5)

(2,1) - (2,2) * (2,3) - (2,4) * (2,5) → for(1-5)

(3,1) - (3,2) - (3,3) * (3,4) - (3,5) → for(1-5)

(4,1) - (4,2) * (4,3) - (4,4) * (4,5) → for(1-5)

* - - - * (5,1) (5,2) (5,3) (5,4) (5,5) → for(1-5)

→ leftside

* = (1,1) (2,2), (3,3) (4,4) (5,5)

Right side

* = (1,5) = 6

(2,4) = 6

(3,3) = 6

(4,2) = 6

(5,1) = 6

i=1

1≤5

↳ 1≤5

↳ i==j || i+j == n+1 : 5+1 = 6

↳ *

2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ - ↳ - ↳ - ↳ *

2≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ - ↳ * ↳ - * ↳ -

3≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ - ↳ - ↳ * ↳ - ↳ -

4≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ - ↳ * ↳ - ↳ * ↳ -

5≤5

↳ 1≤5 2≤5 3≤5 4≤5 5≤5 6≤5 ×

↳ * ↳ - ↳ - ↳ - ↳ *

Q.4.

(1,1) (1,2) (1,3) (1,4) (1,5)
* * * * * → for(1-5)
(2,1) (2,2) (2,3) (2,4)
* * * * → for(1-4)
(3,1) (3,2) (3,3)
* * * → for(1-3)
(4,1) (4,2)
* * → for(1-2)
(5,1)
* → for(1-1)

last step = (1,5) = 6 , (2,4) = 6, (3,3) = 6, (4,2) = 6
(5,1) = 6

i=1 → ~~i++~~ (1,1) = 2, (1,2) = 3, (1,3) = 4, (1,4) = 5, (1,5) = 6
i=2 → (2,1) = 3, (2,2) = 4, (2,3) = 5, (2,4) = 6
i=3 → (3,1) = 4, (3,2) = 5, (3,3) = 6
i=4 → (4,1) = 5, (4,2) = 6
i=5 → (5,1) = 6.

if $\boxed{i+j <= n+1 \quad (5+1) = 6}$

Timecomplexity

```
int n = 5;
for(int i = 1; i <= n; i++){ → [1,n]
    for(int j = 1; j <= n; j++){ → [1,n]
```

(1+n+1+n) * (5+5n+5+5n)
(2n+2) * (10n+10)
$20n^2 + 20n + 20n + 20$
$20n^2 + 40n + 20$
Remove constant
$20n^2 + 40n$
Remove lower value
$20n^2$
Consider, growth rate

$\boxed{TC = O(n^2) \quad SC = O(1).}$

Q.5

(1,1) (1,2) (1,3) (1,4)
* # * # → for(1-4)
(2,1) (2,2) (2,3) (2,4) → for(1-4)
* # * # → for(1-4)
(3,1) (3,2) (3,3) (3,4) → for(1-4)
* # * → for(1-4)
(4,1) (4,2) (4,3) (4,4)

* = (1,1) (1,3) (2,2) (2,4) (3,1) (3,3) (4,2) (4,4)
↳2 ↳4 ↳4 ↳6 ↳4 ↳6 ↳6 ↳8

= (1,2) (1,4) (2,1) (2,3) (3,2) (3,4) (4,1) (4,3)
↳3 ↳5 ↳3 ↳5 ↳5 ↳7 ↳5 ↳7

(i+j) % 2 == 0 → even → *
(i+j) % 2 != 0 → odd → #

1 ≤ 4
↳ 1 ≤ 4 ↳ 1+1 = 2 ↳ *
2 ≤ 4 ↳ 1+2 = 3 ↳ #
3 ≤ 4 ↳ 1+3 = 4 ↳ *
4 ≤ 4 ↳ 1+4 = 5 ↳ #
5 ≤ 4 ✗

2 ≤ 4
↳ 1 ≤ 4 ↳ 2+1 = 3 ↳ #
2 ≤ 4 ↳ 2+2 = 4 ↳ *
3 ≤ 4 ↳ 2+3 = 5 ↳ #
4 ≤ 4 ↳ 2+4 = 6 ↳ *
5 ≤ 4 ✗

3 ≤ 4
↳ 1 ≤ 4 ↳ 3+1 = 4 ↳ *
2 ≤ 4 ↳ 3+2 = 5 ↳ #
3 ≤ 4 ↳ 3+3 = 6 ↳ *
4 ≤ 4 ↳ 3+4 = 7 ↳ #
5 ≤ 4 ✗

4 ≤ 4
↳ 1 ≤ 4 ↳ 4+1 = 5 ↳ #
2 ≤ 4 ↳ 4+2 = 6 ↳ *
3 ≤ 4 ↳ 4+3 = 7 ↳ #
4 ≤ 4 ↳ 4+4 = 8 ↳ *
5 ≤ 4 ✗

5 ≤ 4 ✗

Time complexity

```
int n = 4;
for(int i=1; i≤n; i++){        → (1,n)
    for(int j=1; j≤n; j++){    → (1,n)
```

$(1+n+1+n) * ((4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2+16n+16n+16$

$16n^2+32n+16$

Remove constant

$16n^2+32n$

Remove lower value

$16n^2$

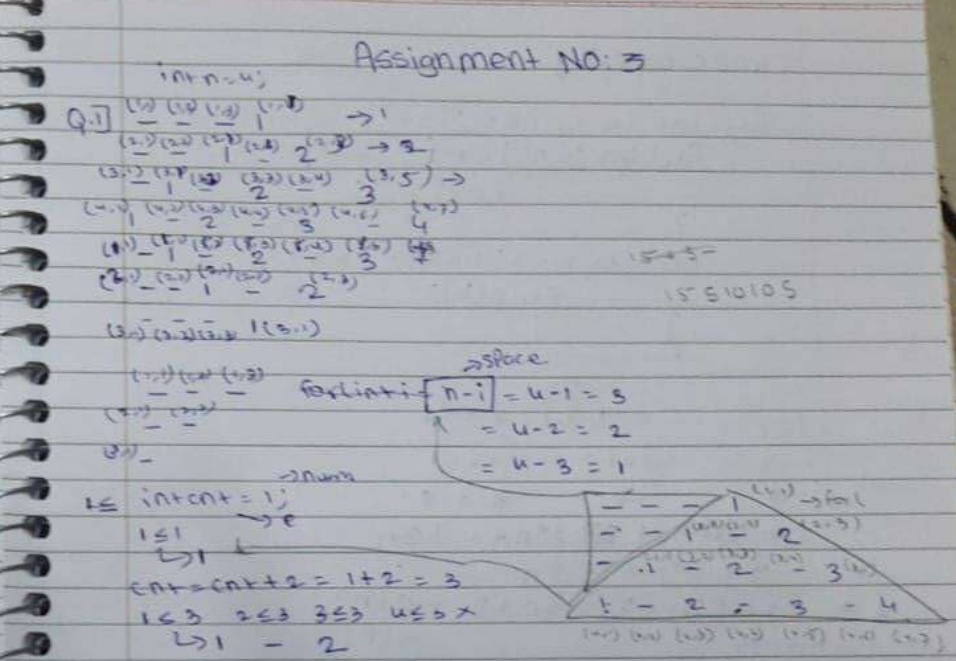
Consider, worst case

TC =	$O(n^2)$
SC =	$O(1)$

Assignment No: 3

int n = 4;

Q.1]



for(int i = n-i) = 4-1 = 3
= 4-2 = 2
= 4-3 = 1

→ space

→ num

int cnt = 1;

1≤ → e

1≤1 → 1

cnt = cnt+2 = 1+2 = 3

1≤3 2≤3 3≤3 4≤3 ×

cnt = cnt+2 = 3+2 = 5

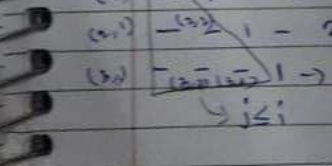
cnt = cnt+2 = 5+2 = 7

Number Print = (1,1), (2,1), (2,3), (3,1) (3,3) (3,5) (4,1) (4,3) (4,5), (4,7)

= (2,2), (3,2), (3,4), (4,2), (4,4), (4,6)

4 5 7 6 8 10

i%2 == 0 i+j%2 == 0 i%2 == 0 i+j%2 == 0



for(i-5)

int n = 4-1; 3

n

colum = 2*n-1
= 6-1 = 5

for(i-3)

for(i-1)

j≤i

~~5-3=~~ ~~5-3-1~~

5-3 = 2

3-1 = 2

Time complexity

```
int n = 5;                                  → [1, n]
for (int i = 1; i = n; i++){                → [1, n]
    for(int j = 1; j = n; j++){
```

$(1 + n + 1 + n) * (5 + 5n + 5 + 5n)$

$(2n + 2) * (10n + 10)$

$20n^2 + 20n + 20n + 20$

$20n^2 + 40n + 20$

Remove constants

$20n^2 + 40n$

Remove Lower value

$20n^2$

consider, growth rate

$TC = O(n^2)$

$SC = O(1)$

Q.3

1	*	–	–	–	–	→ for(i=1) → 4
2	*	*	–	–	–	→ for(i=2) → 4
3	*	*	*	–	–	→ for(i=3) → 4
4	*	*	*	*	–	→ for(i=4) → 4
5	*	*	*	*	*	→ for(i=5) → 4

i=1
↳ 1≤5
↳ 1≤1 ↳ * 2≤1 ×

2≤5
↳ 1≤2 ↳ * 2≤2 ↳ 4 3≤2 ×

3≤5
↳ 1≤3 ↳ * 2≤3 ↳ 4 3≤3 ↳ 4 4≤3 ×

4≤5
↳ 1≤4 ↳ 4 2≤4 ↳ 4 3≤4 ↳ 4 4≤4 ↳ 4 5≤4 ×

5≤5
↳ 1≤5 ↳ * 2≤5 ↳ * 3≤5 ↳ * 4≤5 ↳ * 5≤5 ↳ 4 6≤5 ×

5 ×

Timecomplexity

```
int n = 5;                                  → [1, n]
for(int i = 1; i ≤ n; i++){                 → (1+2+3+4+...+n)
    for(int j = 1; j ≤ i; j++){
```

i = 1	j = 1
i = 2	j = 1, 2
i = 3	j = 1, 2, 3
i = 4	j = 1, 2, 3, 4
i = 5	j = 1, 2, 3, 4, 5
⋮	⋮
n	n

$(1 + 2 + 3 + 4 + 5 + \dots + n) = \frac{n(n+1)}{2}$

$= \frac{n^2 + n}{2}$

$= \frac{n^2}{2} + \frac{n}{2}$

Remove Lower value

$= \frac{n^2}{2}$

$TC = O(n^2)$

$SC = O(1)$

Timecomplexity

```
for(int i=1; i<=n; i++){
    for(int j=1; j<=i; j++){
        s.o.p("-");
    }
    for(int m=i; m>=1; m--){

    }
}
```

$\rightarrow (1+2+3+\dots+n) \rightarrow O(n^2)$

$\rightarrow (n+n-1+n-2+\dots+1) \rightarrow O(n^2)$

$\rightarrow O(2n^2)$

$j=1 \quad (1+2+3+\dots+n) = \frac{n(n+1)}{2}$

$i=1$	$j=1$	$m=1$
$i=2$	$j=1,2,$	$m=2$
$i=3$	$j=1,2,3$	$m=3,2,1,$
	$\vee(1+2+3+\dots+n)$	$\vee(1+2+3+\dots+n)$.

$$\frac{n(n+1)}{2} + \frac{n(n+1)}{2} = \frac{n^2+n}{2} + \frac{n^2+n}{2}$$

$$= \frac{n^2}{2} + \frac{n}{2} + \frac{n^2}{2} + \frac{n}{2}$$

$$= \frac{2n^2}{2} + \frac{2n}{2}$$

$$= n^2 + n$$

Remove lower value

$TC = \Theta(n^2)$

$SC = O(1)$.

Q.3

```
(1,1)                 (1,5)
1  -  -  -  1
-  2  -  2  -
-  -  3  -  -
-  4  -  4  -
5  -  -  -  5
(5,1).                (5,5)
```

Number = (1,1) (1,5) (2,2) (2,4) (3,3) (4,2) (4,4) (5,1) (5,5)

$\rightarrow 6$ $\rightarrow 6$

different co-ordinate sum = 6

i == j = no. Print.

if (i == j || i+j = n+1).

int n=5;

~~for~~ i≤n

```
i≤n,
  ↳ j≤n, s.o.p(i);
```

1≤5	2≤5	3≤5	4≤5	5≤5	6≤5 ×
↳1	↳-	↳-	↳-	↳1	

2≤5

↳1≤5	2≤5	3≤5	4≤5	5≤5	6≤5 ×
↳-	↳2	↳-	↳2	↳-	

3≤5

↳1≤5	2≤5	3≤5	4≤5	5≤5	6≤5 ×
↳-	↳-	↳3	↳-	↳-	

4≤5

↳1≤5	2≤5	3≤5	4≤5	5≤5	6≤5 ×
↳-	↳4	↳-	↳4	↳-	

5≤5

↳1≤5	2≤5	3≤5	4≤5	5≤5	6≤5 ×
↳5	↳-	↳-	↳-	↳5	

6≤5 ×

Timcomplexity. n=5

```
for(int i=1; i≤n; i++){
    for(int j=1; j≤n; j++){
```

i=1	j=1,2,3,4
i=2	j=1,2,3,4
i=3	j=1,2,3,4
i=4	j=1,2,3,4
i=5	j=1,2,3,4.

Q.5

-	-	-	1 - for(1-8)
-	2	- 3	→ for(1-8)
- 4	5	6	→ for(1-8)
7	8	9	10 for(1-8)
- 11	- 12	- 13	
-	14	- 15	
-	-	-	16

int n=4;

Space = 4 - i = 4-1 = 3

i=2 = 4-i = 4-2 = 2

i=3 = 4-i = 4-3 = 1

i+2 = 8+2 = 5+2 = 7

↳ 2 add → for loop.

1≤1 → 1

1≤8 → 2 - 3

1≤5 → 4 - 5 - 6

1≤7 → 7 - 8 - 9 - 10

- (2,2), (2,3), (3,4), (4,2), (4,3), (4,4)

↳4 ↳5 ↳7 ↳6 ↳8 ↳10

i==2 sum=4

i==3 sum==5

→ i+j = (3,1) = 4

(2,2) = 4

(1,3) = 4.

Time complexity

```
for(int i=1; i≤n; i++){
    for(int j=1; j≤n-i; j++){
        s.o.p("-");
    }
}
for (int k=1; k≤sent; k++){
}
```

int n=3

i=1. j=1,2 (1+2) = 2 (2+1+0+...+0)

i=2 j=1

i=3 j=0

k=1 → (1+3+5+...+n)

k=1,2,3

k=1,2,3,4,5

$$\left(\frac{n(n+1)}{2}\right) + \left(\frac{n(n+1)}{2}\right)$$

$$\left(\frac{n^2+n}{2}\right) + \left(\frac{n^2+n}{2}\right)$$

$$\frac{n^2+n+n^2+n}{2} = \frac{2n^2+2n}{2} = \frac{2n^2}{2} + \frac{2n}{2} = n^2+n$$

Remove lower value

TC = O(n2)

SC = O(1)

Q.7

-	-	-	1
-	0	1	
1	-	0	- 1
0	- 1	0	- 1
- 1	-	0	- 1
-	0	-	1
-	-	-	1

int n=4;

Space = $\boxed{n-i}$

i=1 = 4-1 = 3

i=2 = 4-2 = 2

i=3 = 4-3 = 1

(i+j) = (1,1) (2,2) (3,1) (3,3) (4,2) (4,3)

0 = (2,1) (3,2), (4,3)

↳3 ↳5 ↳7

i+j = $\boxed{(i+j)\%2 == 1 \quad (i+j)\%2 != 0.}$

(3,1)

(2,1) (2,2)

(1,1) (1,2) (1,3)

(i+j <= n) → Space

3+1 <= 4

2+2 <= 4

1+3 <= 4

Page No.
Date.

Time complexity.

int n = 4

```
for(int i=1; i≤n; i++){
    for(int j=1; j≤n; j++){
        s.o.p("-");
    }

    for(int k=1; k≤i; k++){

    }
```

$(4-1) = 3 \rightarrow (3+2+1+ \cdots (n-1))$

$\rightarrow (1+2+3+4+ \cdots n)$

$i=1$	$j = 1, 2, 3,$		$k = 1$
$i=2$	$j = 1, 2$	$(4-2) = 2$	$k = 1, 2$
$i=3$	$j = 1$	$4-3 = 1$	$k = 1, 2, 3$
$i=4$	$j =$	$\cancel{4-4=0}$	$k = 1, 2, 3, 4$

$$\left(\frac{n(n+1)}{2}\right) + \left(\frac{n(n+1)}{2}\right) = \frac{n^2+n}{2} + \frac{n^2+n}{2} = \frac{2n^2+2n}{2}$$

$\frac{2n^2}{2} + \frac{2n}{2} = n^2+n$ Remove lower value

$TC = O(n^2)$

$SC = O(1)$

Page No:
Date:

Regular Lecture

1]

O O O O
O O O O
O O O O
O O O O

```
int n = 4;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++);
        S.O.P(0);
    }S.O.Pln();
}
```

Timecomplexity

i = 1	j = 4
i = 2	j = 4
i = 3	j = 4
i = 4	j = 4

$(1+n+1+n) * (4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2 + 16n + 16n + 16$

$16n^2 + 32n + 16$

Remove constant

$16n^2 + 32n$

Remove lower vale

$16n^2$

~~consider growth rate~~

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

2]

(1,1) ~~(1,2)~~ (1,3) (1,4)
O O O O
(2,1) (2,2) (2,3) O (2,4)
(3,1) (3,2) (3,3) O (3,4)

```
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n+1; j++){
        if(i == 2 && j == 3){
            S.O.P("_");
        }else{
            S.O.P(0);
        }
    }S.O.Pln();
}
```

Timecomplexity

i = 1	j = 4
i = 2	j = 4
i = 3	j = 4

$(1+n+1+n) * (3+n+3n)$

$(2n+2) * (4n+3)$

$8n^2 + 3n + 8n + 6$

$8n^2 + 11n + 6$

Remove constant & Remove lower value

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

3]

O	O	O	O	O
O	-	-	-	O
O	-	-	-	O
O	O	O	O	O
O	O	O	O	O

```
for(int i=1; i<=n; i++){
    for(int j=1; j<=n; j++){
        if(i>=2 && i<=3 &&
          j>=2 && j<=4){
            s.o.p("_");
        }else{
            s.o.p(0);
        }
    }s.o.p(n);
```

i=1 j=5
i=2 j=5
i=3 j=5
i=4 j=6
i=5 j=5

$(1+n+1+n) * (5+5n+5+5n)$

$(2n+2) + (10n+10)$

$20n^2+20n+20n+20$

$20n^2+40n+20$

remove constant & lower value

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

4]

(1,1) *

(2,1) * * (2,2)

(3,1) * - * (3,3)

(4,1) * - - * (4,4)

(5,1) * * * * * (5,5)

- = (3,2) (4,2) (4,3)

* = (1,1) (2,1) (2,2) (3,1) (3,3) (4,1) (4,4) (5,1) (5,2) (5,3) (5,4) (5,5)

if (j==1 || j==i || i==n)

i=1 j=1
i=2 j=2
i=3 j=3
i=4 j=4
i=5 j=5

$(1+2+3+4+5+\dots+n) = \frac{n(n+1)}{2}$

$= \frac{n^2+n}{2}$

$= \frac{n^2}{2} + \frac{n}{2}$

~~TC=0~~ =

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

Page No:
Date.

5)

```
O = = = O
= O = O =
= = O = =
= O = O =
O = = = O
```

```
int n = 5;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++){
        if (i+j == n+1 || j == i){
            s.o.p(O);
        }else{
            s.o.p("=");
        }
    }s.o.p.ln();
}
```

i = 1 j = 1, 2, 3, 4, 5
i = 2 j = 1, 2, 3, 4, 5
i = 3 j = 1, 2, 3, 4, 5
i = 4 j = 1, 2, 3, 4, 5
i = 5 j = 1, 2, 3, 4, 5

$(1+n+1+n) * (5+5n+5+5n)$
$(2n+2) * (10n+10)$
$20n^2 + 20n + 20n + 20$
$20n^2 + 40n + 20$

Remove constant & lower value

$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$

6)

1	2	3
4	5	6
7	8	9

```
int n = 3;
int cnt = 1;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++){
        s.o.p(cnt);
        cnt++;
    }s.o.p.ln();
}
```

i = 1 j = 1, 2, 3
i = 2 j = 1, 2, 3
i = 3 j = 1, 2, 3
j = 4 ×

$(1+n+1+n) * (3+3n+3+3n)$
$(2n+2) * (6n+6)$
$12n^2 + 12n + 12n + 12$
$12n^2 + 24n + 12$

Remove constant & lower value

$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$

$(1+n+1+n) * (4+4(n+1)+4n)$

$(2n+2) * (4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2+16n+16n+16$

$16n^2+32n+16$

Remove constants

$16n^2+32n$

Remove lower value

$16n^2$

consider, growth rate

$TC = O(n^2)$

$SC = O(1)$

Q4

(1,1) 3	(1,2) 3	(1,3) 3	(1,4) 3	(1,5) 3
(2,1) 3	(2,2) 2	(2,3) 2	(2,4) 2	(2,5) 3
(3,1) 3	(3,2) 2	(3,3) 1	(3,4) 2	(3,5) 3
(4,1) 3	(4,2) 2	(4,3) 2	(4,4) 2	(4,5) 3
(5,1) 3	(5,2) 3	(5,3) 3	(5,4) 3	(5,5) 3

int n = 3;

int col = 3 + (n-1)

= 3 + (3-1)

= 3 + 2 = 5

e.g.:- (2,2) co-ordinate no prints

~~5-1 = 5~~

2 → no

top = $3-2 = 1$ (n-i)

left = $3-2 = 1$ (n-i)

bottom = ~~(n-(2*i))-1(2n-1+1)~~ ~~i*2=4~~

~~i=2 → 2*2-1 = 4-1 = 3~~ 2n

~~i=3 → 2*3-1 = 6-1 = 5~~

i=1 $(2n-i+1) = 6-1+1 = 6-2 = 4$

i=2 $(2n-i+1) = 6-2+1 = \cancel{6+1=5}\ 6-3 = 3$

right = 2n - 1 - j

j=2 = 2×3 - 1 - 2 = 6 - 3 = 3

layer = n - smallest (left, right, top, bottom)

s.o.p (n - layer no);

Timecomplexity:-

```
for(int i=1; i<=num; i++){
    for(int j=1; j<=num; j++){
```

num = 3

i = 1 j = 1, 2, 3

i = 2 j = 1, 2, 3

i = 3 j = 1, 2, 3

$(1+n+1+n) * (3+3n+3+3n)$

$(2n+2) * (6n+6)$

$12n^2+12n+12n+12$

$12n^2+24n+12$

Remove, constants

$12n^2+24n$

Remove lower value

$12n^2$

consider, growth rate

$TC = O(n^2)$

$SC = O(1)$

11)

```
(1,1) (1,2) (1,3) (1,4)
*     *     *     *
(2,1) (2,2) (2,3) (2,4)
      *     *     *
(3,1) (3,2) (3,3) (3,4)
            *     *
(4,1) (4,2) (4,3) (4,4)
                  *
```

i = 1 j = 1, 2, 3, 4
i = 2 j = 1, 2, 3, 4
i = 3 j = 1, 2, 3, 4
i = 4 j = 1, 2, 3, 4

```
int n = 4;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++)
        if(j < i){
            s.o.p("-");
        }else{
            s.o.p("*");
        }
    s.o.pln();
}
```

$(1+n+1+n) * (4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2+16n+16n+16$

$16n^2+32n+16$

Remove constant & lower value

$\boxed{TC = O(n^2) \quad SC = O(1)}$

12)

```
- - - *
- - * * *
- * * * * *
* * * * * * *
```

i =	j = (n-i)	k =
1	4-1 = 3	1
2	4-2 = 2	3
3	4-3 = 1	5
4	4-4 = 0	7

```
int n = 4;
int cnt = 1;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n-i; j++){
        s.o.p("-");
    }
    for(int k = 1; k ≤ cnt; k++){
        s.o.p("*");
    }
    cnt += 2;
    s.o.pln();
}
```

$4(3+2+1+\cdots n)$ $4(1+3+5+7+\cdots+n)$

$\left(\frac{n(n+1)}{2}\right) + \left(\frac{n(n+1)}{2}\right)$

$\left(\frac{n^2+n}{2}\right) + \left(\frac{n^2+n}{2}\right) = \frac{2n^2+2n}{2} = \frac{2n^2}{2} + \frac{2n}{2}$

Remove lower value

$\boxed{TC = O(n^2) \quad SC = O(1)}$

13)

```
- *
- * - *
* - * - *
```

i = 1, 2, 3

$\boxed{TC \quad SC}$

14)

```
*
(2,1) -
(3,1) -
```

i = 1
i = 2
i = 3

13) - *
- - * - *
* - * - *

i =	j = n - i	k
1	3 - 1 = 2	1
2	3 - 2 = 1	2
3	3 - 3 = 0	3

```
int n = 3; int cnt = 1;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n - i; j++){
        S.o.p("-");
    }
    for(int k = 1; k ≤ cnt; k+=2){
        S.o.p("*" + " ");
    } cnt++;
    S.o.pln();
}
```

$\downarrow (2+1+0 \ldots (n-1)) \rightarrow (1+2+3+\ldots+n)$

$$\left(\frac{n(n+1)}{2}\right) + \left(\frac{n(n+1)}{2}\right)$$

$$\left(\frac{n^2+n}{2}\right) + \left(\frac{n^2+n}{2}\right)$$

$$\frac{2n^2+2n}{2} = \frac{2n^2}{2} + \frac{2n}{2} = n^2+n$$

Remove lower value

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

14) * * * * *
(2,1) * * *
(3,1) — (3,3) *

i = 1	j = 0	k = 1, 2, 3, 4, 5
i = 2	j = 1	k 1, 2, 3
i = 3	j = 2	k = 1

$(0+1+2+\ldots+n)$ $\downarrow (5+3+1+\ldots+n)$

```
int n = 3;
int cnt = 3 + (3 - 1) = 5
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j < i; j++){
        S.o.p("-");
    }
    for(int k = 1; k ≤ cnt; k+=2){
        S.o.p("*");
    } cnt-= 2;
    S.o.pln();
}
```

$$\left(\frac{n(n+1)}{2}\right) + \left(\frac{n(n+1)}{2}\right)$$

$$\frac{2n^2+2n}{2} = \frac{2n^2}{2} + \frac{2n}{2} = n^2+n$$

Remove lower values.

$$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$$

9)

(1,1) _	(1,2) _	(1,3) _	(1,4) *
(2,1) _	(2,2) _	(2,3) *	(2,4) *
(3,1) _	(3,2) *	(3,3) *	(3,4) *
(4,1) *	(4,2) *	(4,3) *	(4,4) *

```
int n = 4;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++){
        if (i+j > n+1){
            s.o.p("*");
        }else{
            s.o.p("_");
        }
    }s.o.pln();
}
```

i = 1 j = 1, 2, 3, 4
i = 2 j = 1, 2, 3, 4
i = 3 j = 1, 2, 3, 4
i = 4 j = 1, 2, 3, 4

$(1+n+1+n) * (4+4n+4+4n)$
$(2n+2) * (8n+8)$
$16n^2 + 16n + 16n + 16$
$16n^2 + 32n + 16$
Remove constant & lower value

$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$

10)

(1,1) *	(1,2) *	(1,3) *	(1,4) *
(2,1) *	(2,2) *	(2,3) *	_
(3,1) *	(3,2) *	_	_
(4,4) *	_	_	_

```
int n = 4;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++){
        if (i+j <= n+1){
            s.o.p("*");
        }else{
            s.o.p("_");
        }
    }s.o.pln();
}
```

i = 1 j = 1, 2, 3, 4
i = 2 j = 1, 2, 3, 4
i = 3 j = 1, 2, 3, 4
i = 4 j = 1, 2, 3, 4

$(1+n+1+n) * (4+4n+4+4n)$
$(2n+2) * (8n+8)$
$16n^2 + 16n + 16n + 16$
Remove constant & lower value

$\boxed{\begin{array}{l} TC = O(n^2) \\ SC = O(1) \end{array}}$

15]

```
int n=36;
for(int i=1; i<= Math.sqrt(n); i++){
        if (n%i==0){
                s.o.p(i);
                ~~s.o.p(n/i);~~
                if(i!=(n/i)){
                        s.o.p(n/i);
}
}
}
```

→ [1, √n]

i=1 i<6
i=1 ~~i=6~~ 1
i=2 2
i=3 3
i=4 4
i=5 5
i=6 6

∴ [1, √n] → total no of iteration decrease as compare with n.

~~[1+2+3+4+5+6+.....+√n]~~

~~$\frac{\sqrt{n}(\sqrt{n}+1)}{2}$~~

$$\boxed{\begin{array}{l} TC = O\sqrt{n} \\ SC = O(1) \end{array}}$$

16)

```
int a=20;
int b=28; int gcd=1
for(int i=1; i<=a; i++){
        if(a%i==0 && b%i==0){
                gcd=i;
}
}
s.o.p(gcd);
```

→ (1, 20] ↳ constant no.

$$\boxed{\begin{array}{l} TC = O(\min(a,b)) \\ SC = O(1) \end{array}}$$

17] int

TC
SC

18) int
wh

?
Ste
1
2
3
4

17)
```
int intgcd=1;
    while(a!=0 && b!=0){
        if (a>b){
            a= a%b;
        }else{
            b=b%a;
        }
    }
    if(a==0){
        gcd=b;
    }else{
        gcd=a;
    }
}
```

$$\boxed{\begin{array}{l} TC = (\log \text{ Math.min}(a,b)) \\ SC = O(1) \end{array}}$$

18)
```
int n = 1234;
while(n>0){
    int rem = n%10;
    S.O.P(rem);
    n=n/10; -> divin.
}
```

$10^k > 0$

$\log_{10} 4 > 0$

$k \log_{10}$

Steps	n	n%10	n/10
1	1234	1234%10 = 4	1234/10 = 123
2	123	123%10 = 3	123/10 = 12
3	12	12%10 = 2	12/10 = 1
4	1	1%10 = 1	1/10 = 0.

$$\boxed{\begin{array}{l} TC = \log_{10} n \\ SC = O(1) \end{array}}$$

7)

1	2	3
1	2	3
1	2	3

```
int n = 3;
for(int i=1; i≤n; i++){
    int cnt = 1;
    for(int j=1; j≤n; j++){
        S.o.p(cnt);
        cnt++;
    }S.o.pln();
}
```

i = 1 j = 1, 2, 3
i = 2 j = 1, 2, 3
i = 3 j = 1, 2, 3.

$(1+n+1+n) * (3+3n+3+3n)$

$(2n+2) * (6n+6)$

$12n^2+12n+12n+12$

$12n^2+24n+12$

Remove constant & lower value

$\boxed{TC = O(n^2) \quad SC = O(1)}$

8)

1	1	1
1	1	1
1	1	1

```
int n = 3;
for(int i=1; i≤n; i++){
    for(int j=1; j≤n; j++){
        int cnt = 1;
        S.o.p(cnt);
        cnt++;
    }S.o.pln();
}
```

i = 1 j = 1, 2, 3
i = 2 j = 1, 2, 3
i = 3 j = 1, 2, 3

$(1+n+1+n) * (3+3n+3+3n)$

$(2n+2) * (6n+6)$

$12n^2+12n+12n+12$

$12n^2+24n+12$

Remove constant & lower value

$\boxed{TC = O(n^2) \quad SC = O(1)}$

Time complexity

n = 3

```
for(int i = 1; i ≤ n; i++){          → [1, n]
    for(int j = 1; j ≤ n-i; j++){    → [1, n-i]
        s.o.p("-");
    }
    for(int k = 1; k ≤ cnt; k++){    → [1, cnt]
        s.o.p(k);
    }
}
```

$(1+n+1+n)$ $\quad$ ~~(3+~~ $\quad$ $3-1=2$

$i=1$ $\quad$ $j=1,2,3$ $\quad$ $3-1=2$

$i=2$ $\quad$ $j=1,2$ $\quad$ $3-2=1$

$i=3$ $\quad$ $j=0,1$ $\quad$ $3-3=0$

$2*n-1$

$2*3+1=6+1$

$2(3+1)$

$2(4)=8$

~~(3+~~ $2*n+1+3n$

~~$3+2*n+1+3n$~~

$j =$ ~~$2n+3n+4=5n+4=O(n)$~~

$O(n)$

~~$k=3$~~

$i=1$ $\quad$ $k=1,2$

$i=2$ $\quad$ $k=1,2,3,4$

$i=3$ $\quad$ $k=1,2,3,4,5,6$

~~(3+~~

$n+n=2n\times n=2n^2$

$O=(n^2)$

$TC = O(n^2)$

$SC = O(1)$

Q2.

1	–	–	–	–	–	–	1
1	2	–	–	–	–	2	1
1	2	3	–	–	3	2	1
1	2	3	4	4	3	2	1
1	2	3	4	4	3	2	1
1	2	3	–	–	3	2	1
1	2	–	–	–	–	2	1
1	–	–	–	–	–	–	1

break the Problem

1	1	–	–	–	–	–	–	1
2	1	2	–	–	–	–	2	1
3	1	2	3	–	–	3	2	1
4	1	2	3	4	4	3	2	1

int n = 4;

~~$2*4-1=8$~~

~~$2*(n-1+1)$~~ spaces

~~$i-1=2*(4-1+1)=8$~~

$i=1$ = 6 spaces -2

$i=2$ = 4 spaces -2

$i=3$ = 2 spaces -2

$i=4$ = 0 spaces

$\boxed{2*(n-i)} = 2(4-1) = 6$

$= 2(4-2) = 4$

$= 2(4-3) = 2$

$= 2(4-4) = 0$

$i=1$ for(1-1)

2 for(1-2)

3 for(1-3)

4 for(1-4).

↳ $(j \le i)$

for

~~$j=i;\ j>$~~ $(j=i;\ j\ge 1;\ j--)$

$1\ge 1$ ↳1 $\quad$ $2\ge 1$ ↳2 $\quad$ $1\ge 1$ ↳1

4	1	2	3	4	4	3	2	1
3	1	2	3	–	–	3	2	1
2	1	2	–	–	–	–	2	1
1	1	–	–	–	–	–	–	1

→ space = 0 → reverse the outer loop.

= 2 +2

= 4 +2

= 6 +2

$16n^2 + 32n + 16$

Remove constants

$16n^2 + 32n$

Remove lower value

$16n^2$

consider, growth rate

$TC = O(n^2)$

$SC = O(1)$

Q.3

(1,1)	(2,2)	(1,3)	(1,4)	
1	2	3	4	→ for(1-4)
(2,1) 1	(2,2) 2	(2,3) 3	4 (2,4)	→ for(1-4)
(3,1) 1	(3,2) 2	(3,3) 3	4 (3,4)	→ for(1-4)
1 (4,1)	2 (4,2)	3 (4,3)	4 (4,4)	→ for(1-4)

$1*1,\ 1+1=2,\ 2+1=3,\ 3+1=4,\ 4+1=5^{\times}$

int cnt = 1

i=1 $1 \le 4$

↳ $1 \le 4$ ↳ 1 $1+1=2$ | $2 \le 4$ ↳ 2 $2+1=3$ | $3 \le 4$ ↳ 3 $3+1=4$ | $4 \le 4$ ↳ 4 $4+1=5$ | $5 \le 4 \rightarrow \times$

cnt = 1

i=2 $2 \le 4$

↳ $1 \le 4$ ↳ 1 $1+1=2$ | $2 \le 4$ ↳ 2 $2+1=3$ | $3 \le 4$ ↳ 3 $3+1=4$ | $4 \le 4$ ↳ 4 $4+1=5$ | $5 \le 4 \times$

cnt = 1

$3 \le 4$

↳ $1 \le 4$ ↳ 1 $1+1=2$ | $2 \le 4$ ↳ 2 $2+1=3$ | $3 \le 4$ ↳ 3 $3+1=4$ | $4 \le 4$ ↳ 4 $4+1=5$ | $5 \le 4 \times$

cnt = 1

$4 \le 4$

↳ $1 \le 4$ ↳ 1 $1+1=2$ | $2 \le 4$ ↳ 2 $2+1=3$ | $3 \le 4$ ↳ 3 $3+1=4$ | $4 \le 4$ ↳ 4 $4+1=5$ | $5 \le 4 \times$

$5 \le 4 \rightarrow$ false.

Time complexity:

```
int n = 4;
for(int i=1; i≤n; i++){        → (1,n)
    for(int j=1; j≤n; j++){    → (1,n)
```

$(1+n+1+n) * (4+4n+4+4n)$

$(2n+2) * (8n+8)$

$16n^2 + 16n + 16n + 16$

$16n^2 + 32n + 16$

Remove constants

$16n^2 + 32n$

Remove lower value

$16n^2$

consider, growth rate

$TC = O(n^2)$

$SC = O(1)$.